Automated Reasoning and Knowledge Inference on OPC UA Information Models

J. Bakakeu, M. Brossog, J. Zeitler and J. Franke
Institute for Factory Automation and Production Systems
Friedrich-Alexander-Universität Erlangen-Nürnberg
Egerlandstr. 7-9, 91058 Erlangen, Germany
Email: jupiter.bakakeu@faps.fau.de

S. Tolksdorf, H. Klos and J. Peschke
Digital Factory Division
Siemens AG
Gleiwitzer Str. 555, 90475 Nürnberg, Germany
Email: hans-henning.klos@siemens.com

***Abstract*—The fourth industrial revolution demands flexibility, adaptability, transparency and semantic interoperability. Within the German Industry 4.0 initiative, the Reference Architecture Model Industrie 4.0 (RAMI4.0) has recently been standardized and OPC Unified Architecture (OPC UA) is listed as the sole recommendation for implementation of a communication layer. Even though OPC UA helps bridge the interoperability gap at the automation level, its semantic has not yet been formally defined and an efficient automated reasoning and knowledge inference on the OPC UA address space is therefore not yet possible. This paper addresses this issue by presenting a solution to infer knowledge from OPC UA information models. By analyzing and comparing the semantic expressiveness of the OPC UA address space with semantic knowledge representation formalisms such as RDF and OWL, we derived and implemented a solution to transform an OPC UA information model into an RDF-Graph expressing an OWL Ontology. Using the generated ontology, we were able to run a true reasoning task directly on the OPC UA address space. In this paper, the developed approach is conveniently validated on various case studies involving online factory reconfiguration, intelligent energy management, and human-machine interfaces.**

***Index Terms*—OPC UA; Sementic Web; Knowledge Inference; Industrie 4.0; SPARQL.**

I. Introduction

The fourth industrial revolution demands flexibility, adaptability, transparency, and semantic interoperability in order to meet today's production environment challenges characterized by frequent changes in terms of high product variation, small batch sizes, high demand fluctuation and random unexpected disturbances on the factory floor [1], [2]. New production systems that can timely adapt their architecture and functionality to these fluctuations are needed [3], [4]. Within the German Industry 4.0 initiative [5], the Reference Architecture Model Industrie 4.0 (RAMI4.0) [6], [7] has been recently standardized and the concept of Reconfigurable Manufacturing Systems (RMS) [8] is recommended as a proper way to achieve the required flexibility.

An RMS considers a modern manufacturing system as a combination and collaboration of cyber-physical assets (Industrie 4.0 components) that offer different capabilities or skills. These self-describing assets can be added, changed and removed at runtime to quickly respond to changing requirements. The core concept relies on the unified and formalized abstraction and description of the manufacturing capabilities (skills provided by the available assets) and the manufacturing tasks (production steps needed) as well as the intelligent orchestration of the different assets that takes into account technical and economic conditions to best meet the defined manufacturing goals. The skill-based system model enables the transition from generic high-level descriptions to low-level formats that can be directly executed as functions within machines.

In this context, a well formalized and uniform information model of the different assets is required. For an orchestration of two or more I4.0-components, it is necessary to automatically access and interpret the information model of the different assets without human intervention. Furthermore, it should be possible to apply a reasoning task on these models in order to check the consistency and to infer new facts that have not been explicitly expressed. In other words, the information model of the assets should be expressed or should be able to be transformed into a formal language based on first-order logic, e.g. a semantic, so that mathematical operations can be applied to extract knowledge.

Many attempts to realize such RMS can be found in the literature [8], [9]. Many of them describe the asset capabilities directly in an ontologies, e.g. semantic web ontologies, and thus take advantage of existing powerful reasoners. However, most ontology technologies and reasoners do not thrive well at the plant floor level, as they were simply not designed to meet the performance and scalability requirements of modern industrial control systems. The difficulty here lies in the synchronization of the ontology server and the reasoner with the physical asset. As a result, the proposed solutions are only applicable in a few use cases with soft timing constraints. For this reason, RAMI4.0 solely recommend OPC Unified Architecture (OPC UA) [10] for the implementation of the communication layer of such flexible reconfigurable manufacturing systems [6], [7], [11], since OPC UA can be used for both, communication and information modeling at the PLC level. Even though OPC UA helps bridge the interoperability gap at the automation level, its semantic has not yet been formally defined and automated reasoning as well as knowledge inference on the OPC UA address space is therefore not yet possible.

978-1-5386-8500-6/19/$31.00 ©2019 IEEE

This paper tries to fill this gap by presenting a solution to infer knowledge directly from OPC UA information models. In our solution, we first analyzed the semantic expressiveness of the OPC UA address model and compared it with standard semantic knowledge representation formalisms such as RDFS and OWL. We proved that an OPC UA information model can always be transformed in an ontology. We then derived and implemented a solution to transform an arbitrary OPC UA information model into an RDF-Graph expressing an OWL Ontology. Using the generated ontology, we were able to run a true reasoning task directly on the OPC UA address space. The developed approach is conveniently verified and validated on the case study of consistency management in multi-viewpoint system engineering.

The rest of the paper is structured as follows: Section II introduces the used technologies and describes the topics under consideration. Section III makes a comparison between web ontologies and the OPC UA information models and leads over to Section IV, which describes the methodology, the architecture, and the developed core components. The next Section verifies and validates the proposed solutions on various case studies involving online factory reconfiguration, intelligent energy management, and human-machine interfaces. The paper ends with a conclusion and an outlook on future work and extensions.

II. Background & Related Works

In this Section, we give a short overview of OPC UA as information modeling for Industrie 4.0, existing well established semantic technologies like the Semantic Web as glue for homogeneous knowledge representation, and description logics as logical formalism for ontologies.

A. OPC UA – Information modeling in Industry 4.0

The OPC Unified Architecture (OPC UA) is a platform independent service-oriented architecture, which is established in industrial environments to communicate and exchange information between different machines [12]. In contrast to conventional solutions, OPC UA presents four major improvements: platform independence, interoperability on the transport layer, a PKI-based security model and the ability to specify and deploy a generic information model.

The platform independency of OPC UA lies in the fact that the protocol is software-based and located on the upper layer (5-7) of the ISO/OSI Model. This makes OPC UA independent of the physical hardware of the participating devices. In addition, custom applications can be built on top of OPC UA. In this case, the interaction between the application and the protocol implementation is established via the OPC UA Stack, which has been standardized and open-sourced in many programming languages by the OPC Foundation, making OPC UA also application independent.

The interoperability on the transport layer is reached through standardization of different transport protocols like OPC TCP and HTTP(S) combined with various encodings like OPC Binary, OPC XML, and OPC JSON. By taking advantage of these well-defined transport protocols, security mechanisms like authentication, authorization, and encryption have been implemented. In the newest release of OPC UA, the well-known publish-subscribe pattern was also introduced, which enables cloud connectivity based on transport protocols like AMQP or MQTT.

Besides the secure transport of information, information modeling is another big fundament of OPC UA [12], [13]. Each OPC UA server includes an information model that allows users to organize data and their knowledge in a structured manner. The information model which constitutes the address spaces of an OPC UA server supports object-oriented model paradigms. Features like classes and entities (Object Types and Objects), properties (mandatory or optional), attributes and methods can be modeled in OPC UA. OPC UA also supports composition and inheritance mechanisms. In addition, the OPC UA information model also foresees references, which identify how nodes are related to each other and generally describe a relationship between two nodes, such as “Node A contains Node B”. As a result, it is possible to build an arbitrary information model expressing domain-specific knowledge on top of each device in an industrial environment. Fig. 1 presents the building blocks of the OPC UA information model.

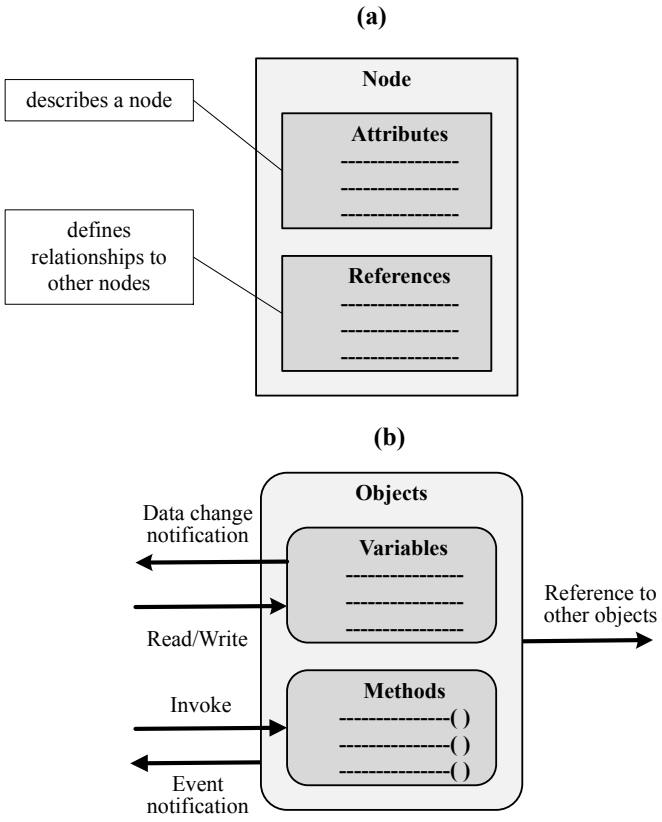


Fig. 1. (a) OPC UA node structure: attributes describe a node while references describe relationships with other nodes. (b) OPC UA object structure: objects are composed of variables/properties and methods. While an external system can read/write object variables/properties and invoke an object method, objects can also send data or event notifications to external systems.

By browsing the references of its parent node starting from

the default root node of the OPC UA server of a device, a "smart" OPC UA client can identify other devices, discover the data structures, the functionalities, and services they offer. Invoking a service from another device can be done by the means of method calls. Through subscriptions, devices can monitor the state and data of other devices or sub-systems and therefore, initiate actions based on these changes without the need for human intervention. The possibilities opened up by OPC UA best suit the vision of a cyber-physical production system, where each system or component can describe itself and optimize its behavior and structure depending on predefined global objectives.

B. Semantic technologies – Glue for homogeneous knowledge representation

The concept of semantic technologies emerged in the early 1960s as a form to represent semantically structured knowledge in a way that they can be understandable by machines as well as humans. It extends a network of hyperlinked human-readable information by inserting machine-readable metadata that specifies how pieces of information are linked with each other and to concepts of a knowledge domain [14]. The Semantic Web [15] is one of the most important technologies that have been developed to implement this vision.

Within the Semantic Web context, the Resource Description Framework (RDF) [16] is the commonly used data model to represent data and the relationship between them. It is a framework that enables the creation of statements in a form of so-called triples. A triple has three components: subject, predicate and object. It represents an assertion of the relationship between subject and object. Here, relationships are directed from the subject to the object and named as a predicate. Triples that refer to the same subjects or objects form a semantic network and thereby store knowledge about the subject that can be analyzed by suitable algorithms. Fig. 2 illustrates a graph representation of two RDF triples.

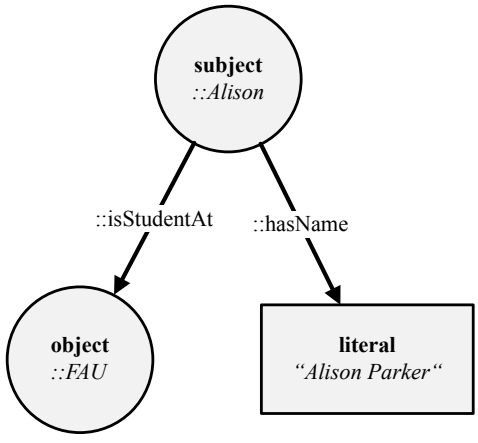


Fig. 2. Graph representation of two RDF triples. The subject Alison is a student at the FAU and at the same time has the full name "Alison Parker"

RDF on its own doesn't give meaning to data. RDF is a data model, i.e. a method to express relationships. To give meaning, vocabularies are required. The RDF Schema (RDFS) [16] provides a basic vocabulary for RDF. Using RDFS it is, for example, possible to create hierarchies of classes and properties. The Web Ontology Language (OWL) [17] extends RDFS by adding more advanced constructs to describe the semantics of RDF statements. It allows stating additional constraints, such as for example cardinality, restrictions of values, or characteristics of properties like transitivity. It is based on description logic and so brings reasoning power to the semantic web. To increase the expressiveness and thus the reasoning capability, OWL can be extended with the Semantic Web Rule Language (SWRL) [18]. SWRL rules are formulas that are applied to the instances, not to classes. SWRL does not only provide simple rules but also built-ins. They can be used for example to handle strings or mathematical calculations [18].

Along with data modeling and vocabularies, the semantic web technologies also define query mechanisms and query languages. SPARQL [19] for example is an RDF query language that can be used to query any RDF-based data (i.e., including statements involving RDFS and OWL).

C. Description Logics – logical formalism for ontologies

Description Logics (DL) [20] are classes of logic-based knowledge representation languages that can be used to represent terminological knowledge of an application domain in a formal and structured way. It is considered as a fragment of first-order-logic, but with the particularity that it is decidable and thus enables the deduction of implicit knowledge by inference from a knowledge base. This is the main reason why DLs are commonly used in artificial intelligence to describe and reason about the relevant concepts of an application domain. In the context of semantic technologies, ontology languages like OWL DL [21] have a well-defined and structured syntax that can be easily mapped into a description logic such as SHOIN(D) [22]. This description logic therefore provides the logical formalism for reasoning and knowledge inference tasks on top of OWL-based ontologies.

III. Semantic Expressiveness of OPC UA

As presented in Section II-A, the OPC UA address space can be considered as a directed graph where the nodes (OPC UA nodes) are for example objects or data points and the edges (OPC UA references) are for relations between the nodes. In this context, the information model of an OPC UA server can be viewed as a triple store that holds triples in the form of "subject - predicate - object", where a triple represents an assertion in which subject and object are related. Here, relationships are directed from the subject to the object and named predicate. Triples that refer to the same subjects or objects form a semantic network and thereby store knowledge. Therefore, it is possible to form one or more semantic networks (knowledge) into the information model of an OPC UA server. An OPC UA information model can consequently be considered as a valid knowledge representation model.

If the predicates of an OPC UA information model represented by OPC UA references have mathematical properties such as inheritance, transitivity, reflexivity, domain, range, cardinality, etc., then it is possible by means of mathematical methods to analyze the encoded semantic networks of an OPC UA information model and thus, evaluate the expressiveness of OPC UA as knowledge representation model. Per default, the OPC UA specification defines a set of reference types (”HasSubType”, ”HasType”, HasProperty ”, etc.) as an integral part of the standardized address space model. The mathematical properties of these reference types are by definition implicitly defined. For example, the reference ”HasSubType”, which describes the inheritance property, is also transitive. In addition, all reference types have default attributes that provide additional information about their mathematical properties. For example, the “Symmetric” attribute is used to indicate whether the meaning of the reference type is the same for the source and destination nodes.

In order to compare or to categorize OPC UA with other well-known knowledge representation models, we defined an ordering relationship on knowledge representation model called *expressive* as follow: a knowledge representation model is as *expressive* as another if it can encode all the meanings of the other model. We can state for example that, higher-order logics are more *expressive* than first-order logics, which in turn are more *expressive* than description logics. The operator *expressive* gives a partial order on knowledge representation models because some models might encode some of the statements of one language but not others.

Using these ordering, we can state that OPC UA is more *expressive* than a taxonomy since it is possible to model any taxonomy, which is a semantic hierarchy in which information entities are related to each other by subsumption relations, in an OPC UA information model using the reference “HasSubClass”. OPC UA is also at least as expressive as a conceptual model since it has been shown that conceptual models like UML can be modeled in an OPC UA information model [23], [24].

Unfortunately, the OPC UA specification lacks the ability to explicitly define assertions or axioms on references in addition to the implicitly defined mathematical properties of reference types that are present in the specification. Furthermore, the specification doesn’t specify all reference types needed to transform an OPC UA information model to a description logic like it is the case with OWL DL. As an example, minimum and maximum cardinality, domain, or range of an OPC UA reference cannot be explicitly modeled. In conclusion, we can state that OPC UA is less expressive than OWL DL. Fig. 3 shows the alignment of OPC UA as knowledge representation in the ontology spectrum as defined by Obrst et al. [25] and Table I presents an excerpt of axioms or properties of OWL that cannot be expressed in OPC UA.

IV. Proposed Approach

This Section aims to present our solution for reasoning and knowledge inference on OPC UA information models.

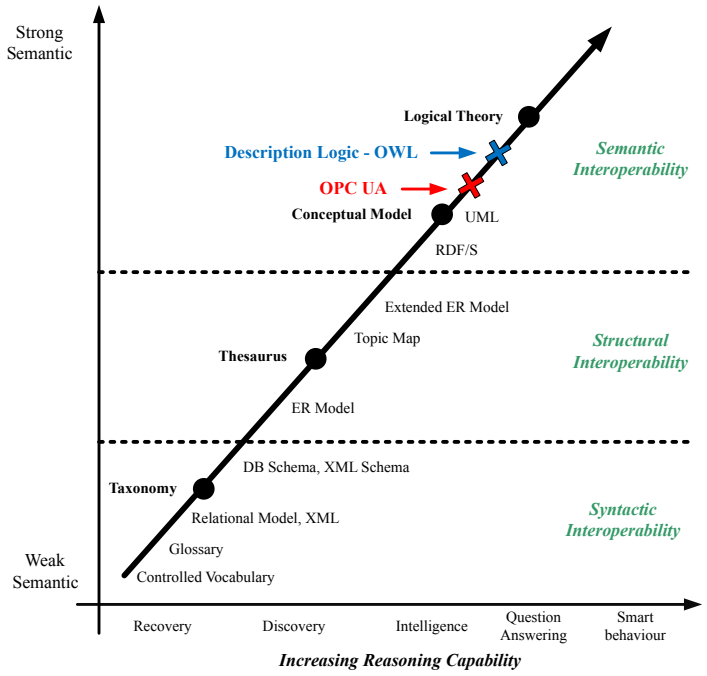


Fig. 3. Alignment of OPC UA as knowledge representation in the semantic spectrum [25]. OPC UA is more expressive than a conceptual model and less expressive than OWL.

TABLE I
EXAMPLES OF OWL AXIOMS AND PROPERTIES THAT CANNOT BE MODELED USING OPC UA [17].

Axioms/Properties	OWL Elements
Data intersection	DataIntersectionOf
Data range	DataRange
Classes equivalence	EquivalentClasses
Disjoint classes	DisjointClasses
Object property subsumption	SubObjectPropertyOf
Object property equivalence	EquivalentObjectProperties
Object property domain	ObjectPropertyDomain
Object property range	ObjectPropertyRange
Object property reflexivity	ReflexiveObjectProperty
Data property range	DataPropertyRange
Data property subsumption	SubDataPropertyOf
Data property equivalence	EquivalentDataProperties
Individual equality	SameIndividual

Our approach first transforms the knowledge encoded by an OPC UA information model into a formal ontology (OWL DL) and then uses existing reasoners and techniques to infer new knowledge. This Section also presents a prototype and its architecture that exposes the encoded knowledge on a SPARQL server.

A. The Method

In Section III, we defined a partial order on knowledge representation models and demonstrate that OPC UA is a

valid knowledge representation model with more expressiveness than taxonomies and conceptual models but with less expressiveness than ontologies like OWL (e.g. OWL DL). This implies that an ontology like OWL DL can encode all the knowledge represented in an OPC UA information model. Therefore a transformation of an OPC UA information model into an ontology, that preserves the encoded knowledge, can always be found, but the opposite will require additional information and constraints on the ontology side.

Our approach takes advantage of this fact and proposes a transformation of the concepts (entities and relations) of the standard OPC UA namespace (http://opcfoundation.org/UA/) to an OWL ontology (OWL DL). Since every OPC UA information model by definition uses or extends this namespace, the proposed transformation is applicable to all OPC UA information models. In the target ontologies we took advantage of existing modeling concepts like "SubClassOf" to map OPC UA references like "HasSubType" and we extend the ontology with SWRL rules, properties, and classes to map complex OPC UA elements like "SymetricReference" and "HasEventSource". Fig. 4 presents the concepts used to model the standard OPC UA namespace and Table II presents an excerpt of the transformation rules used.

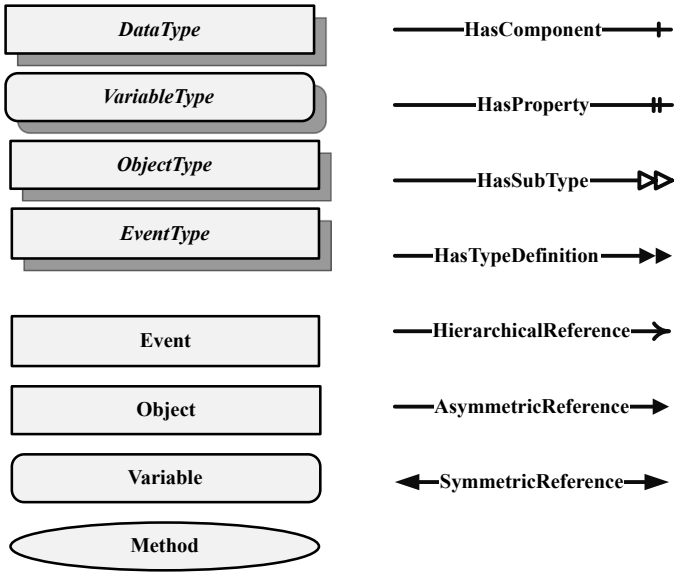


Fig. 4. Concepts of the standard OPC UA namespace as OPC UA notation for address space modeling. These concepts constitute the basis for the transformation of the OPC UA address space to OWL DL

By transforming an OPC UA information model into an ontology, we encoded the knowledge it contains in a formal ontology on which complex mathematical operations can be applied. This makes it possible to perform complex queries on the information model and automatically apply logical thinking to generate new knowledge. This is very important since the state-of-the-art provides many reasoners and techniques to analyze the transformed information.

As a consequence, a "smart" OPC UA client can browse the complete information model of an OPC UA server, transform it into an ontology and then use well-established reasoners like Pellet [26] or FACT++ [27] and query mechanisms like SPARQL to extract or infer the knowledge needed at runtime. With this powerful ability, production machines can unambiguously extract data and contextual information, interpret the retrieved information correctly, reason about these information, and act accordingly.

TABLE II
EXCERPT OF THE TRANSFORMATION RULES USED TO MAP OPC UA CONCEPTS INTO OWL CONCEPTS.

OPC UA Concept	**OWL Concept**
ObjectType, EventType Class HasSubType	SubClassOf
HasProperty	RDF Property
HasTypeDefinition	RDF Type
HasComponent	RDF Member
Object, Method, Reference, View	Individual
Attribute "NodeId"	Extended RDF resource uri constructed with the namespace uri.
Attribute "Datatype"	Datatype or Class
Attribute "Value"	DataHasValue
Attribute "Description"	Literal
...	...

B. The Prototype

In order to demonstrate the feasibility and the usability of our solution for knowledge inference and reasoning on OPC UA information model, we implemented a model transformer as a software solution that uses an OPC UA client to browse the complete address space of an OPC UA server, applies our transformation rules to convert the extracted information model into an OWL DL ontology and then exposes the transformed knowledge via a SPARQL endpoint.

As illustrated in Fig. 5, the model transformer has 5 components:

- An *OPC UA client* that establishes a connection to an OPC UA server, browses the information model and builds an internal graph representation of the server address space. The client, which is implemented using eclipse Milo [28], also synchronizes the internal graph representation with the information model of the OPC UA Server. If new events are generated or a value of a data variable changes, the client updates the internal graph so that it remains aligned with the server address space.
- A *Triple-Store Adapter* contains the transormation rules defined in Section IV-A and applies them on the internal graph to generate OWL DL compliant triples that encode the knowledge exposed by the OPC UA server. The Triple- Store Adapter is implemented using the framework Jena [29], [30].

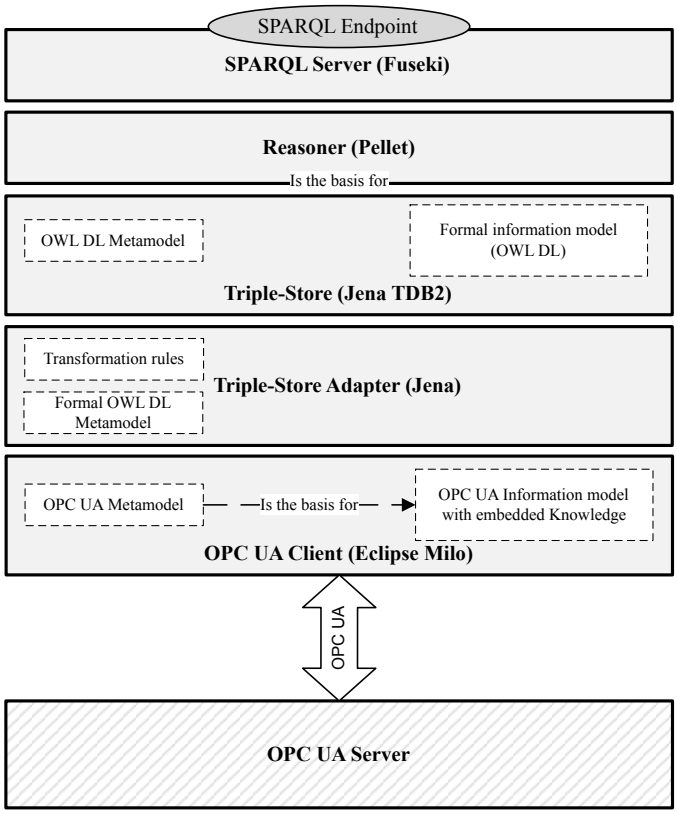
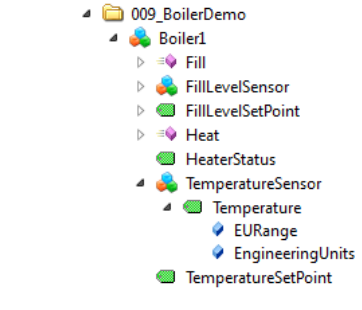


Fig. 5. Architecture of the OPC UA to OWL DL converter.

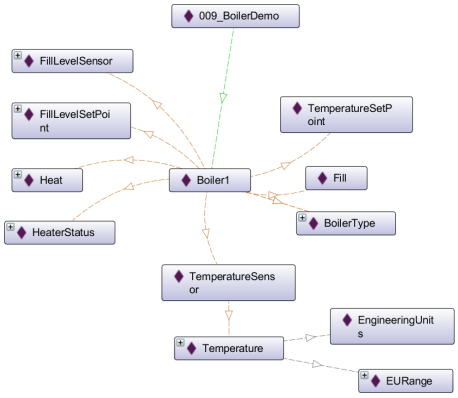
- In order to store the generated triples, the Jena Tuple DataBase *TDB2* [30] is used. This triple-store holds all the semantic graphs generated from the OPC UA information model.
- On top of the Triple-Store TDB, a *reasoner* (Pellet [26]) analyses the generated sematic graph and infers new triples, if necessary.
- In order to expose the inferred knowledge, a SPARQL server *Fuseki* [30] is integrated.

With the presented architecture, complex queries, reasoning, and knowledge inference requests in form of SPARQL queries can be directly executed on the transformed information model. Since the resulting OWL ontology only formalizes the knowledge modeled by the address space of an OPC UA server, the results of these queries will only express the knowledge exposed by the OPC UA server. Therefore, the presented model transformer constitutes a tool for automated reasoning and knowledge inference on OPC UA information models.

Fig. 6 shows the example of an OPC UA information model (a) transformed into OWL DL (b) and a SPARQL query verifying that all objects of type “BoilerType” have a temperature sensor.



(a)



(b)

```
PREFIX opcua:           <http://opcfoundation.org/UA/>
PREFIX rdf:             <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs:            <http://www.w3.org/2000/01/rdf-schema#>
PREFIX opcuauserns1:    <urn:eclipse:milo:hello-world>
ASK {
    ?boiler opcua:N40 ?boilertype .
    ?boilertype   rdfs:label "BoilerType" .
    ?y opcua:N47 ?temp_sensor .
    ?temp_sensor opcua:N40 ?temp_sensor_type .
    ?temp_sensor_type   rdfs:label "TemperatureSensorType"
  }
```

(c)

Fig. 6. Example of an OPC UA information model (a) transformed into OWL DL (b) and a SPARQL query asserting that all objects of type “BoilerType” have at least one temperature sensor.

V. Case Studies

The following case studies highlight the potential of automated reasoning and knowledge inference on OPC UA information model. The presented use cases have been implemented on the Industrie 4.0 Demonstrator of the Institute for Factory Automation and Production Systems of the University of Erlangen-Nurnberg.

The Industrie 4.0 demonstrator is a flexible cyber-physical production system that assembles and packs bottles of different shape, size, capsule colors, and contents. It has 3 modules: a modular conveyor system consisting of 3 flexible conveyor segments with 4 assembling position each, a portal robot that handles the Pick-and-Place operations, and a gripper station with different grippers (see Fig. 7). The conveyor segments are modular and present similar electrical and mechanical interfaces so that they can be reconfigured and repositioned in order to meet the actual production process requirements. The conveyor segments offer services that expose their actual configuration and actual state as well as services to move a carrier from one assembling position to another. They are also

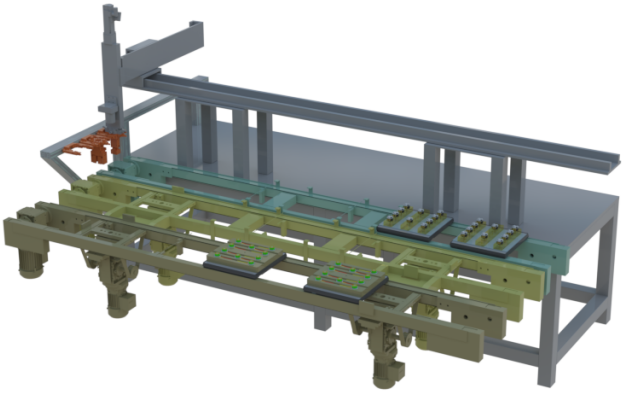


Fig. 7. The Industrie 4.0 demonstrator on which the different scenarios have been implemented.

equipped with RFID-readers to read/write the RFID-chip of the carriers, where the product configurations and the results of the previous production task are written. The portal robot can reach all assembling positions of the conveyor segments as well as every gripper of the gripper station. It is intelligent in the sense that it can compute the number and order of Pick-and-Place operations required to assemble the products such that predefined goals like energy consumption are optimized. While executing a Pick-and-Place operation, the robot computes a movement path that optimizes its energy consumption as well as its dynamic behavior. It is also equipped with a camera module that inspects the result of every operation. If an operation failed, it can reinitiate or cancel it. The gripper station offers different gripper types and presents the available gripper type as a service. Depending on the task, the portal robot decides which gripper is more appropriate. Each module and sub-module of the demonstrator is equipped with a separate PLC so that they can expose their own information and services. The PLCs (soft PLC) are equipped with both an OPC UA server to expose the capabilities and services of the submodule they control, and a model transformer as implemented in Section IV-B to access and interpret the information provided by surrounding modules. Thanks to a cloud connection, a production controller located at the cloud (on-premise) monitors and oversees the production process.

Case-Study 1 - Adaptable Factory: The control and the configuration of production systems has to be performed quickly and reliably. If a new gripper with for example a wider claw width or a new material is introduced, the portal robot should be able to identify the new gripper and its properties. Traditionally the operator himself has to reconfigure the portal robot and reprogram the production tasks to use the new well-suited gripper. By accessing the OPC UA server of the gripper station, where the properties of the gripper are listed and by transforming these properties into a formal ontology, the portal robot can automatically analyze the properties of the new gripper just by issuing SPARQL requests. Similarly each time the production system is reconfigured, one can access the description of all modules and check if their functional interfaces are compatible and if a production task is realizable by the actual system configuration. If this is not the case, the operator can rearrange or introduce new production modules.

Case-Study 2 – Energy Management: Today resource and energy efficiency are of increasing relevance in the production environment. Traditional solutions for energy optimization will record the energy consumed by every module during the production, build a complex model of the production system offline and use a solver to compute fixed energy management strategies that are then deployed on the production system. In the context of flexible production systems with individualized products, these approaches are too expensive, since a reconfiguration of the production system will imply redesigning the optimization strategies. If the modules of the production system can describe their energetic behavior, then an intelligent system can interpret this information on-the-fly and optimize the energy consumption of the machines. This approach is flexible and cost-efficient. It has been successfully realized at the Industrie 4.0 demonstrator [31], [32].

Case-Study 3 – Human-Machine-Interaction: The required flexibility of future production systems will make it very difficult to implement a user interface that is suitable for all possible configurations of the production system. This can only be realized, if the user interface is smart enough to extract knowledge of the production environment at runtime, to interpret this knowledge and present it to the operator. Furthermore, future human-machine-interfaces should be able to match the knowledge they have gained from the production system with the request of the user. By transforming the information exposed by the production system into a formal ontology, user requests can be transformed into a SPARQL request, which can then be summited to the SPARQL endpoint of the model transformer implemented in Section IV-B. This scenario has been implemented on the Industrie 4.0 demonstrator. Here we used a natural language interface to drive the operator during a diagnosis task. The advantage for operators is that they can communicate naturally with the machine, independently of the actual configuration. Moreover, it is not important how familiar the operator is with the machine: the system can answer questions in colloquial language.

All these case-studies require knowledge to perform their tasks in high quality. To validate whether or not our solution for automatic knowledge inference and reasoning on OPC UA information model is suitable, the use cases illustrated above were implemented in the Industrie 4.0 demonstrator. The particularity of our implementation is that no additional hardware or software was required. All the needed knowledge was encoded on the OPC UA information model of the PLCs of all modules.

VI. Summary and Outlook

In this paper, we highlighted the importance of semantic interoperability on the factory floor and showed that OPC UA is a valid knowledge representation model and hence a suitable candidate to realize the vision of re-configurable

manufacturing systems. Then we compared the semantic expressiveness of the OPC UA address space with other well-known knowledge representation formalisms. By introducing a partial ordering on knowledge representation models, we found that OPC UA is more expressive than taxonomies or conceptual models, but it lacks mechanisms and concepts that will make it at least as expressive as a description logic. In our case, we compared it with OWL DL. Based on these findings, we design a solution to transform an OPC UA information model into an OWL DL Ontology expressing the same knowledge. This transformation has the advantage that the knowledge presented in an OPC UA information model can be transformed into a formal language based on first-order logic, on which automated reasoning and knowledge inference can be applied. Last but not least, we presented a prototype, to show the feasibility of our concept. We also presented several benefits of our architecture in various case studies involving online factory reconfiguration, intelligent energy management, and human-machine interfaces.

REFERENCES

[1] S. Hu, X. Zhu, H. Wang, and Y. Koren, "Product variety and manufacturing complexity in assembly systems and supply chains," *CIRP Annals*, vol. 57, no. 1, pp. 45 – 48, 2008.

[2] H.-P. Wiendahl, H. ElMaraghy, P. Nyhuis, M. Zäh, H.-H. Wiendahl, N. Duffie, and M. Brieke, "Changeable manufacturing - classification, design and operation," *CIRP Annals*, vol. 56, no. 2, pp. 783 – 809, 2007.

[3] D. Mourtzis and M. Doukas, "Decentralized manufacturing systems review: challenges and outlook," *Logistics Research*, vol. 5, no. 3, pp. 113–121, Nov 2012.

[4] B. Vogel-Heuser, C. Diedrich, A. Fay, S. Jeschke, S. Kowalewski, M. Wollschlaeger, and P. Göhner, "Challenges for software engineering in automation," *Scientific Research*, May 2014. [Online]. Available: https://www.scirp.org/journal/PaperInformation.aspx?PaperID=46096

[5] H. Kagermann, W. Wahlster, and J. Helbig, "Recommendations for implementing the strategic initiative industrie 4.0: Securing the future of german manufacturing industry," 2013.

[6] "Industrie 4.0: The reference architectural model industrie 4.0 (rami 4.0)," ZVEI, Tech. Rep., April 2015.

[7] "Status report-reference architecture model industrie 4.0 (rami4.0)," VDI/VDE-Gesellschaft Mess- und Automatisierungstechnik, Tech. Rep., June 2015.

[8] J. Pfrommer, D. Stogl, K. Aleksandrov, S. E. Navarro, B. Hein, and J. Beyerer, "Plug and produce by modelling skills and service-oriented orchestration of reconfigurable manufacturing systems," *at - Automatisierungstechnik*, vol. 63, no. 10, jan 2015.

[9] M. Loskyll, J. Schlick, S. Hodek, L. Ollinger, T. Gerber, and B. Pîrvu, "Semantic service discovery and orchestration for manufacturing processes," in *ETFA2011*, Sep. 2011, pp. 1–8.

[10] *IEC 62541 - OPC Unified Architecture*, OPC Foundation Std., Nov 2015.

[11] Y. Li, S. Hu, W. Tao, and B. Li, "Research on the industrial network architecture of northbound interface," in *2017 Chinese Automation Congress (CAC)*, Oct 2017, pp. 6505–6509.

[12] W. Mahnke, S.-H. Leitner, and M. Damm, *OPC Unified Architecture*. Springer, 2009.

[13] J. Bakakeu, F. Schäfer, J. Bauer, M. Michl, and J. Franke, *Building Cyber-Physical Systems – A Smart Building Use Case*. John Wiley & Sons, Ltd, 2017, ch. 21, pp. 605–639. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119226444.ch21

[14] T. Berners-Lee, J. Hendler, and O. Lassila, "The semantic web," *Scientific American*, no. 284, pp. 34–43, may 2001.

[15] "Resource description framework (rdf): Concepts and abstract syntax," W3C, Feb 2004. [Online]. Available: https://www.w3.org/TR/rdf-concepts/

[16] "Rdf semantics," W3C, Feb 2004. [Online]. Available: https://www.w3.org/TR/rdf-mt/

[17] "Owl 2 web ontology language document overview (second edition)," W3C, Dec 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/

[18] I. H. et al., "Swrl: A semantic web rule language combining owl and ruleml," W3C, May 2004. [Online]. Available: https://www.w3.org/Submission/SWRL/

[19] E. Prud'hommeaux and A. Seaborne, "Sparql query language for rdf," W3C, Jan 2008. [Online]. Available: https://www.w3.org/TR/rdf-sparql-query/

[20] L. F. Sikos, *Description Logics in Multimedia Reasoning*. Springer, 2017.

[21] L. Ma, Y. Yang, Z. Qiu, G. Xie, Y. Pan, and S. Liu, "Towards a complete owl ontology benchmark," in *The Semantic Web: Research and Applications*, Y. Sure and J. Domingue, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 125–139.

[22] I. Horrocks and P. F. Patel-Schneider, "Reducing owl entailment to description logic satisfiability," in *The Semantic Web - ISWC 2003*, D. Fensel, K. Sycara, and J. Mylopoulos, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 17–29.

[23] B. Lee, D.-K. Kim, H. Yang, and S. Oh, "Model transformation between opc ua and uml," *Computer Standards & Interfaces*, vol. 50, pp. 236 – 250, 2017.

[24] S. Rohjans, K. Piech, and S. Lehnhoff, "Uml-based modeling of opc ua address spaces for power systems," in *2013 IEEE International Workshop on Inteligent Energy Systems (IWIES)*, Nov 2013, pp. 209–214.

[25] M. C. Daconta, L. J. Obrst, and K. T. Smith, *The Semantic Web: A Guide to the Future of XML, Web Services, and Knowledge Management*. Wiley, 2003.

[26] E. Sirin, B. Parsia, B. C. Grau, A. Kalyanpur, and Y. Katz, "Pellet: A practical owl-dl reasoner," *Journal of Web Semantics*, vol. 5, no. 2, pp. 51 – 53, 2007, software Engineering and the Semantic Web. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1570826807000169

[27] D. Tsarkov and I. Horrocks, "Fact++ description logic reasoner: System description," in *Automated Reasoning*, U. Furbach and N. Shankar, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 292–297.

[28] J. Reimann. (2017) Developing opc ua with eclipse milo. [Online]. Available: https://www.eclipsecon.org/europe2017/session/developing-opc-uaeclipse-milo

[29] D. Jeong, H. Shin, D.-K. Baik, and Y.-S. Jeong, "An efficient web ontology storage considering hierarchical knowledge for jena-based applications," *JIPS*, vol. 5, pp. 11–18, 2009.

[30] "What is jena," The Apache Foundation, 2015. [Online]. Available: https://jena.apache.org/

[31] J. Bakakeu, S. Tolksdorf, J. Bauer, H.-H. Klos, J. Peschke, A. Fehrle, W. Eberlein, J. Bürner, M. Brossog, L. Jahn, and J. Franke, "An artificial intelligence approach for online optimization of flexible manufacturing systems," in *Energy Efficiency in Strategy of Sustainable Production IV*, ser. Applied Mechanics and Materials, vol. 882. Trans Tech Publications, 8 2018, pp. 96–108.

[32] T. Javied, J. Bakakeu, D. Gessinger, and J. Franke, "Strategic energy management in industry 4.0 environment," in *2018 Annual IEEE International Systems Conference (SysCon)*, April 2018, pp. 1–4.